Busca

Estruturas de Dados



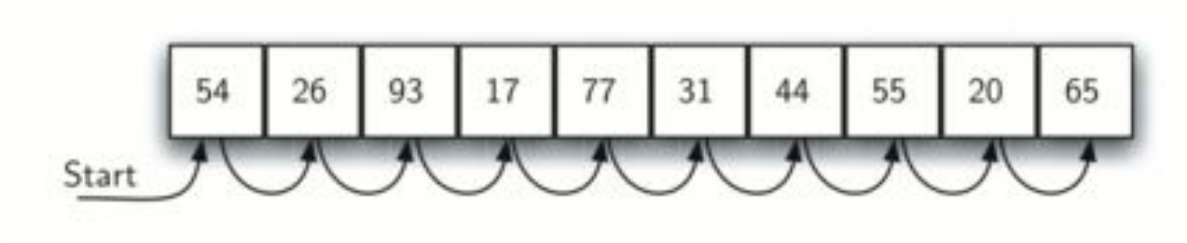
Universidade de Brasília
Departamento de Ciência da Computação

Definição

Busca é o processo algorítmico de encontrar um item específico numa coleção de itens. Uma busca tipicamente devolve True ou False, indicando se o item está presente ou não na coleção.

Busca sequencial

Cada item é armazenado numa posição relativa aos demais. Em listas, essas posições relativas são os índices de cada item.



Busca sequencial com operador in

```
>>> 15 in [3, 5, 2, 4, 1]
False
>>> 3 in [3, 5, 2, 4, 1]
True
```

Busca sequencial

```
def sequencial(alist, item):
    pos = 0
    found = False

    while pos < len(alist) and not found:
        if alist[pos] == item:
            found = True
        else:
            pos = pos + 1

    return found
```

Análise da Busca

A análise dos algoritmos de busca considera:

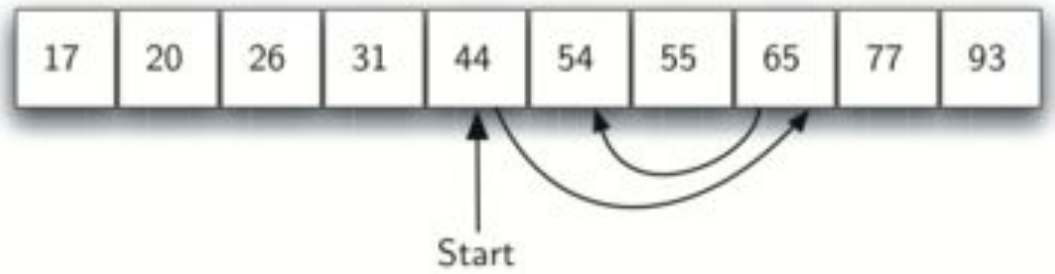
1) O elemento não está na lista.
2) O elemento está na lista:
   a) o item é o primeiro elemento da lista (melhor caso)
   b) o item é o último elemento da lista (pior caso)
   c) o item está mais ou menos na metade da lista (caso médio)

Análise da Busca Sequencial

Caso	Melhor caso	Pior caso	Caso Médio
Item presente	1	n	n/2
Item não presente	n	n	n

Busca Binária

A busca binária em um vetor ordenado começa examinando o item do meio. Se o item procurado for maior que este item, sabemos que a metade inferior não precisa mais ser levada em consideração. O processo se repete até o elemento ser ou não encontrado.



Busca Binária Iterativa

```
def binaria(alist, item):
    first = 0
    last = len(alist) - 1
    found = False
    while first <= last and not found:
        midpoint = (first + last) // 2
        if alist[midpoint] == item:
            found = True
        elif item < alist[midpoint]:
            last = midpoint - 1
        else:
            first = midpoint + 1

    return found
```

Busca Binária Recursiva 1

```
def binaria(alist, item):

    if len(alist) == 0:
        return False
    else:
        midpoint = len(alist) // 2
        if alist[midpoint] == item:
            return True
        elif item < alist[midpoint]:
            return binaria(alist[:midpoint], item)
        else:
            return binaria(alist[midpoint+1:], item)
```

Busca Binária Recursiva 2

```
def binaria(alist, item, inf, sup):

    if sup <= inf:
        return False

    midpoint = (inf + sup) // 2
    if alist[midpoint] > item:
        return binaria(alist, item, inf, midpoint)
    if alist[midpoint] < item:
        return binaria(alist, item, midpoint + 1, sup)

    return True
```

Análise da Busca Binária

Comparações	Número aproximado de itens restantes
1	n/2
2	n/4
3	n/8
…	…
i	$n/2^i$

$$\frac{n}{2^i} = 1$$

$$i = \log n$$

$$O(\log n)$$

Análise da Busca Binária

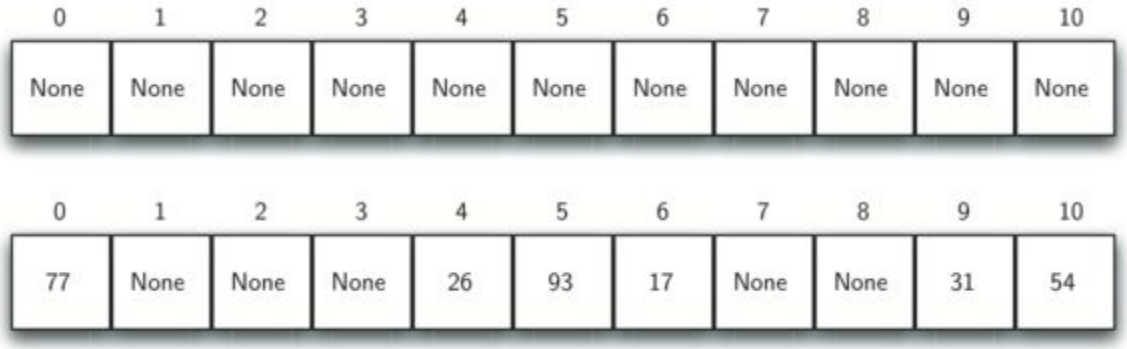
Caso	Melhor caso	Pior caso	Caso Médio
Item presente	1	log n	log n
Item não presente	log n	log n	log n

Tabela de Dispersão

Uma tabela de dispersão (ou hash table) é uma coleção de itens que são armazenados de maneira a serem encontrados com facilidade. Essa estrutura de dados permite busca tem tempo O(1).

Tabela de Dispersão

Uma tabela de dispersão (ou hash table) é uma coleção de itens que são armazenados de maneira a serem encontrados com facilidade. Essa estrutura de dados permite busca tem tempo O(1).



Função de Espalhamento

O mapeamento entre a chave e o índice ao qual ela pertence na tabela é conhecido como a função de espalhamento (ou *hash function*). A função de espalhamento irá receber qualquer item na coleção e irá retornar um inteiro dentro do intervalo dos índices.

Função de Espalhamento

O mapeamento entre a chave e o índice ao qual ela pertence na tabela é conhecido como a função de espalhamento (ou *hash function*). A função de espalhamento irá receber qualquer item na coleção e irá retornar um inteiro dentro do intervalo dos índices.

$$hash(\ item) = item \% \ m$$

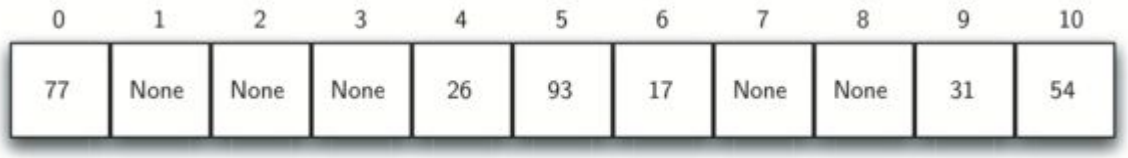
Função de Espalhamento

- Método de folding
  - 436-555-4601
  - 43, 65, 55, 46, 01
  - 43+65+55+46+01 = 210
  - 210 % m
- Método do quadrado do meio
  - 44^2 = 1.936
  - 93 % m

Função de Espalhamento: Colisão!

O que acontece se desejarmos inserir o número 44?

$$hash(\ item) = item \% \ m$$



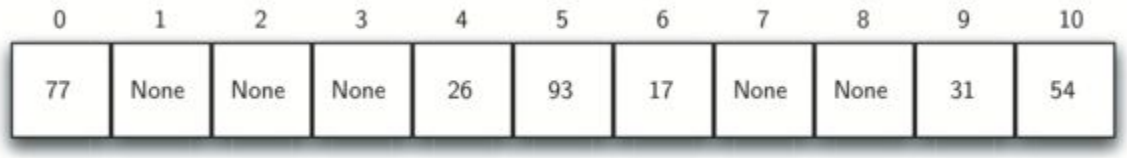
Resolução de colisões

- Endereçamento aberto
  - Sondagem linear
  - Sondagem quadrática
- Encadeamento

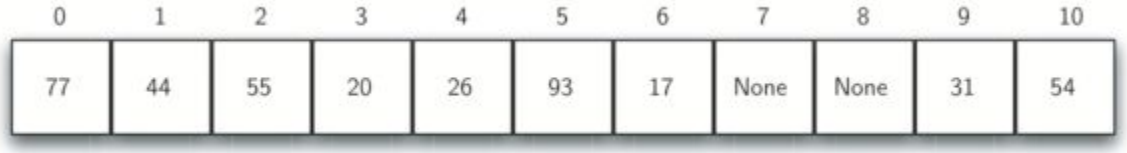
$$hash(item) = item \% m$$

$$rehash(pos) = (pos + 1) \% m$$

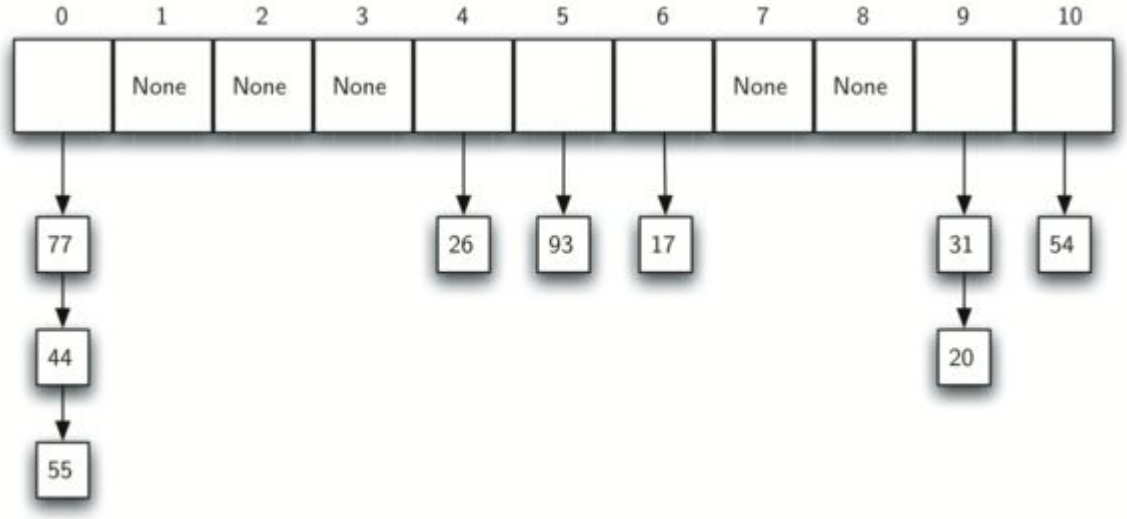
$$rehash(pos) = (pos + pulo) \% m$$



Incluíndo: 44, 55 e 20



Resolução de colisões por encadeamento



Resolução de colisões por encadeamento com dicionário

```
def hashfunction(key, size):
    return key % size

def put(key, size):
    hash[hashfunction(key, size)].append(key)

def get(key, size):
    return key in hash[hashfunction(key, size)]
```

Resolução de colisões por encadeamento com dicionário

```
def hashfunction(key, size):
    return key % size

def put(key, size):
    hash[hashfunction(key, size)].append(key)

def get(key, size):
    return key in hash[hashfunction(key, size)]


hash = {}
for i in range(size):
    hash[i] = []
```

Tipo de Dado Abstrato Mapa

- `Map()` Cria um novo mapa vazio.
- `put(key, val)` Adiciona um novo par chave-valor para o mapa.
- `get(key)` Dada uma chave, retorna o valor armazenado no mapa.
- `del` Elimina o par chave-valor usando uma declaração `del map[key]`
- `len()` Retorna o número de pares chave-valor armazenadas no mapa.
- `in` Retorna `True` se a chave dada estiver no mapa e `False`, caso contrário.

Tipo de Dado Abstrato Mapa

```
class HashTable:

    def __init__(self):
        self.size = 11
        self.slots = [None] * self.size
        self.data = [None] * self.size
```

Tipo de Dado Abstrato Mapa

```
def hashfunction(self, key, size):
    return key % size


def rehash(self, oldhash, size):
    return (oldhash + 1) % size
```

Tipo de Dado Abstrato Mapa

```
def put(self, key, data):
    hashvalue = self.hashfunction(key, len(self.slots))

    if self.slots[hashvalue] is None:
        self.slots[hashvalue] = key
        self.data[hashvalue] = data
    else:
        if self.slots[hashvalue] == key:
            self.data[hashvalue] = data  #replace
        else:
            nextslot = self.rehash(hashvalue, len(self.slots))
            while self.slots[nextslot] is not None and self.slots[nextslot] != key:
                nextslot = self.rehash(nextslot,len(self.slots))

            if self.slots[nextslot] is None:
                self.slots[nextslot] = key
                self.data[nextslot] = data
            else:
                self.data[nextslot] = data #replace
```

Tipo de Dado Abstrato Mapa

```
def get(self, key):
  startslot = self.hashfunction(key, len(self.slots))

  data = None
  stop = False
  found = False
  position = startslot

  while self.slots[position] is not None and not found and not stop:
      if self.slots[position] == key:
          found = True
          data = self.data[position]
      else:
          position=self.rehash(position, len(self.slots))
          if position == startslot:
              stop = True

  return data
```

Tipo de Dado Abstrato Mapa

```
def __getitem__(self,key):
    return self.get(key)

def __setitem__(self,key,data):
    self.put(key,data)
```

Tipo de Dado Abstrato Mapa

```
>>> H=HashTable()
>>> H[54]="cat"
>>> H[26]="dog"
>>> H[93]="lion"
>>> H[17]="tiger"
>>> H[77]="bird"
>>> H[31]="cow"
>>> H[44]="goat"
>>> H[55]="pig"
>>> H[20]="chicken"
>>> H.slots
[77, 44, 55, 20, 26, 93, 17, None, None, 31, 54]
>>> H.data
['bird', 'goat', 'pig', 'chicken', 'dog', 'lion',
       'tiger', None, None, 'cow', 'cat']
```

```
>>> H[20]
'chicken'
>>> H[17]
'tiger'
>>> H[20]='duck'
>>> H[20]
'duck'
>>> H.data
['bird', 'goat', 'pig', 'duck', 'dog', 'lion',
       'tiger', None, None, 'cow', 'cat']
>> print(H[99])
None
```

Resumo

- Implementar uma busca sequencial e binária.
- Entender a ideia de hashing como uma técnica de busca.
- Introduzir o tipo de dado abstrato mapa.
- Implementar o tipo de dado abstrato mapa com *hashing*.
- Implementar uma tabela de dispersão com colisões por encadeamento com dicionário.